

1. WAP to find 2nd smallest element in array

Ans- #include <stdio.h>
void main()

{
int a[50];

int n, min;

min = INT_MAX;

printf("Enter no. of elements\n");

scanf("%d", &n);

printf("Enter the elements\n");

for(int i=0; i<n; i++)

{
scanf("%d", a[i]);

int j=0;

for(int i=0; i<n; i++)

{

if(a[i] < min)

{
min = a[i];

j = i;

}

}

a[j] = INT_MAX;

min = INT_MAX;

```
for(int i=0; i<n; i++)
```

```
{  
    if(a[i] < min)  
        min = a[i];  
}
```

```
printf("2nd smallest element is %d", min);  
}
```

Output:

Enter no. of elements

5

1 2 3 4 5

2nd smallest element is 2

2 ~~#~~ Sum of left diagonal elements

```
#include <stdio.h>
void main()
{
    int a[50][50];
    int n;
    printf("Enter the degree of square matrix");
    scanf("%d", &n);
    int sum = 0;
    printf("Enter the elements\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
            if (i == j)
                sum += a[i][j];
        }
    printf("Sum of left diagonal elements is %d", sum);
}
```

Output:

Enter the degree of square matrix 2

Enter the elements

1	2
3	4

Sum of Left diagonal elements is 5

3)

FCFS

```
#include <stdio.h>
```

```
void main()
```

```
{ int n, i, j;
```

```
printf("Enter no. of processes: ");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n], ct[n], tat[n], wtn[n],  
rt[n], pln[], temp;
```

```
for(i = 0; i < n; i++)
```

```
{ printf("Process P%d\n", i+1);
```

```
printf("Arrival Time: ");
```

```
scanf("%d", &at[i]);
```

```
printf("Burst Time: ");
```

```
scanf("%d", &bt[i]);
```

```
pln[i] = i+1;
```

```
}  
for(i = 0; i < n-1; i++)
```

```
{ for(j = i+1; j < n; j++)
```

```
{ if(at[i] > at[j])
```

```
{
```

```

temp = at[i]; at[i] = at[j]; at[j] = temp;
temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
temp = p[i]; p[i] = p[j]; p[j] = temp;
}

```

```

}

```

```

int currectime = 0;

```

```

for (i = 0; i < n; i++)

```

```

{

```

```

    if (currectime < at[i])

```

```

    {

```

```

        currectime = at[i];

```

```

        rt[i] = currectime - at[i];

```

```

        wt[i] = rt[i];

```

```

        currectime += bt[i];

```

```

        ct[i] = currectime;

```

```

        tat[i] = ct[i] - at[i];

```

```

    }

```

```

printf("In Process \t AT \t BT \t CT \t TAT\n\t WT \t RT\n");

```

```

for (i = 0; i < n; i++)

```

```

    printf("P %d \t %d \t %d \t %d \t %d \t %d\n", p[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

```

```

}

```

```

float total_wt = 0, total_tat = 0, total_rt = 0;
for (i = 0; i < n; i++)

```

```

    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += rt[i];

```

```

printf("Avg WT = %.2f", total_wt / n);
printf("Avg TAT = %.2f", total_tat / n);
printf("Avg RT = %.2f", total_rt / n);
}

```

O/P:

Enter number of processes: 4

Process P₁

Arrival Time: 0

Burst Time: 7

Process P₂

Arrival Time: 0

Burst Time: 13

Process P3

Arrival time = 4 3

Burst time = 4

Process P4

Arrival time = 5

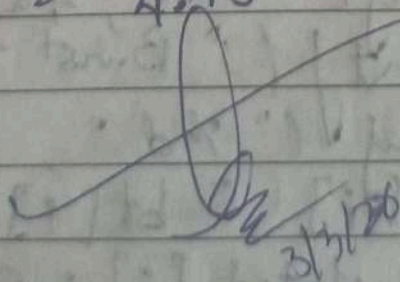
Burst time = 6

Process	AT	BT	CT	TAT	WT	BT
P1	0	7	7	7	0	0
P2	8	3	20	12	9	9
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6

Avg WT = 4.75

Avg TAT = 9.75

Avg RT = 4.75


3/3/20

4. SJF

```
#include <stdio.h>
```

```
void main () {
```

```
    int n, r, choice;
```

```
    printf ("Enter number of processes ");
```

```
    scanf ("%d", &n);
```

```
    int pid[n], at[n], bt[n], rt[n],
```

```
    int ct[n], wt[n], tat[n], resp[n],  
    completed[n];
```

```
    for (i = 0; i < n; i++)
```

```
    {  
        pid[i] = i + 1;
```

```
        printf ("In Process P%d\n", pid[i]);
```

```
        printf ("Arrival Time ");
```

```
        scanf ("%d", &at[i]);
```

```
        printf ("Burst Time ");
```

```
        scanf ("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
        completed[i] = 0;
```

```
        resp[i] = -1;
```

```
    }
```

```

printf ("In Select Scheduling Type: \n" );
printf ("1. Non-Preemptive \n");
printf ("2. Preemptive \n");
printf ("Enter choice: ");
scanf ("%d", &choice);

```

```

int current = 0;

```

```

int complete = 0;

```

```

if (choice == 1)

```

```

while (complete < n)

```

```

{
    int shortest = -1;

```

```

    int min_bt = 10000000;

```

```

    for (i = 0; i < n; i++)
    {

```

```

        if (at[i] <= current && complete[i] == 0)
        {

```

```

            if (bt[i] < min_bt)
            {

```

```

                min_bt = bt[i];

```

```

                shortest = i;
            }

```

```

        else if (bt[i] == min_bt)
        {

```

```

            if (at[i] < at[shortest])
            {

```

```

                shortest = i;
            }
        }
    }
}

```

```
if (shortest == -1)
```

```
{  
    curtime ++;
```

```
else {
```

```
    if (resp[shortest] == -1)
```

```
        resp[shortest] = curtime - at[shortest];
```

```
        curtime += bt[shortest];
```

```
        et[shortest] = curtime;
```

```
        lat[shortest] = et[shortest] - at[shortest];
```

```
        rot[shortest] = tot[shortest] - lat[shortest];
```

```
        completed[shortest] = 1;
```

```
        complete ++;
```

```
else if (choice == 2) {
```

```
    while (complete < n) {
```

```
        int shortest = -1;
```

```
        int min_val = 1000000;
```

```
for (i = 0; i < n; i++) {
    if (at[i] != curtime & & completed[i] == 0)
```

```
    if (at[i] < min-rt) {
        min-rt = at[i];
        shortest = i;
    }
```

```
    else if (at[i] == min-rt) {
        & if (at[i] < at[shortest])
            shortest = i;
    }
```

```
    if (shortest == -1) {
        curtime ++;
```

```
    else {
        if (resp[shortest] == -1)
            resp[shortest] = curtime - at[shortest];
        at[shortest] = -1;
        curtime ++;
```

```
    if (at[shortest] == 0) {
        completed[shortest] = 1;
        complete ++;
```

```

    ct[shortest] = current;
    lat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = lat[shortest] - bt[shortest];
    if (res > lat[i]) {
        res = lat[i];
    }
}

```

```

else {
    printf("Invalid choice ! \n");
    return;
}

```

```

float tot_wt = 0, tot_lat = 0; tot_rt = 0;

```

```

float tot_at;

```

```

printf("In Process \t AT \t BT \t CT\n\t TAT \t WT \t RT \n");

```

```

for (i = 0; i < n; i++)

```

```

    printf("P%d \t %d \t %d \t %d \t %d \t %d \n", pid[i], at[i], bt[i], ct[i], lat[i], wt[i], resp[i]);

```

```

    tot_wt += wt[i];

```

```

    tot_lat += lat[i];

```

```

    tot_rt += resp[i];
}

```

printf ("In Average Waiting Time = %.2f",
total_wt/n);
printf ("In Average Turnaround Time = %.2f",
total_tat/n);
printf ("In Average Response Time = %.2f", total_rt/n);

~~return 0;~~
}

Output:

Enter number of processes : 4

Arrival time for P1 : 0
Burst time for P1 : 7
Arrival time for P2 : 8
Burst time for P2 : 3
Arrival time for P3 : 4
Burst time for P3 : 4
Arrival time for P4 : 5
Burst time for P4 : 6

Enter choice - 1

Non-Preemptive SJF

Process	AT	BT	CT	TAT	WT	RT
P ₁	0	7	4	7	0	0
P ₂	8	3	14	6	3	3
P ₃	3	4	11	8	4	4
P ₄	5	6	20	15	9	9

Avg TAT : 9.00

Avg WT : 4.00

Avg RT : 4.00

Order choice - 2

Preemptive SJF

Process	AT	BT	CT	TAT	WT	RT
P ₁	0	7	7	7	0	0
P ₂	8	3	11	3	0	0
P ₃	3	4	14	11	7	4
P ₄	5	6	20	15	9	9

Avg TAT : 9.00

Avg WT : 4.00

Avg RT : 3.25

Lab - 3

Priority (Non-preemptive, Preemptive),
Round Robin.

#include <stdio.h>

#define MAX 20

/* Priority Non-Preemptive */

void priority_np (int n, int at[], int bt[],
int pn[]) {int wt[MAX] = {0}, tat[MAX], rt[MAX],
ct[MAX], done[MAX] = {0};

int time = 0, count = 0;

float sw = 0, st = 0, sr = 0;

while (count < n) {

int idx = -1, min = 9999;

for (int i = 0; i < n; i++)

if (at[i] <= time && !done[i] && pn[i] < min)

{ min = pn[i];

idx = i;

{

```

if (idx == -1) {
    get[idx] = time - at[idx];
    wt[idx] = time - at[idx];
}

```

```

time += bt[idx];
ct[idx] = time;
tat[idx] = ct[idx] - at[idx];

```

```

done[idx] = 1;
count++;
}

```

```

else
    time++;
}

```

```

printf("\n PID \t AT \t BP \t CT \t WT \t  

\t\t\t TAT \t RT \n");

```

```

for (int i = 0; i < n; i++)

```

```

    printf("%d \t %d \t %d \t %d \t %d \t  

           %d \n", i+1, at[i], bt[i], ct[i],  

           wt[i], tat[i], rt[i]);

```

```

sum += wt[i];
st += tat[i];
avr += rt[i];
}

```

```
printf("\n Avg WT = %.2f \n Avg TAT = %.2f \n Avg RT = %.2f \n",
        s0/n, s1/n, s2/n);
```

/ Priority Preemptive */*

```
void priority-p(int n, int at[], int bt[],
                int pri[]) {
```

```
    int rt_bt[MAX], wt[MAX] = {0}, tat[MAX],
        at[MAX], ct[MAX];
```

```
    for (int i = 0; i < n; i++)
```

```
        rt_bt[i] = bt[i];
```

```
        at[i] = -1;
```

```
}
```

```
    int time = 0, complete = 0;
```

```
    float s0 = 0, s1 = 0, s2 = 0;
```

```
    while (complete < n)
```

```
    {
```

```
        int idx = -1, min = 9999;
```

```
        for (int i = 0; i < n; i++)
```

```
            if (at[i] <= time & rt_bt[i] > 0 & pri[i] < min)
```

```

{ min = p[1];
  idx = 1;
}

```

```

if (idx != -1)
{

```

```

    if (rt[idx] == -1)

```

```

        rt[idx] = time - at[idx];

```

```

        rt[idx] = time - at[idx];
    }

```

```

}

```

```

else

```

```

    time++;

```

```

}

```

```

printf("\n PID \t AT \t BT \t CT \t WT \t TAT \t RT \n");

```

```

for (int i = 0; i < n; i++)
{

```

```

    printf("%d \t %d \t %d \t %d \t %d \n",
           i+1, at[i], bt[i], ct[i], wt[i],
           tat[i], rt[i]);

```

```

    wt += wt[i];

```

```

    tat += tat[i];

```

```

    rt += rt[i];
}

```

/* Round Robin */

```
void rr (int n, int at[], int bt[], int q)
{
    int rt_bt[MAX], wt[MAX] = {0}, lat[MAX],
    et[MAX], ct[MAX];
    int done[MAX] = {0};
    for (int i = 0; i < n; i++)
    {
        rt_bt[i] = bt[i];
        wt[i] = -1;
    }

    int time = 0, complete = 0;
    float sw = 0, st = 0, r = 0;

    while (complete < n)
    {
        for (int i = 0; i < n; i++)
        {
            if (at[i] <= time && rt_bt[i] > 0)
            {
                if (wt[i] == -1)
                {
                    rt_bt[i] = q;
                }
            }
            else
            {
                time += rt_bt[i];
            }
        }
    }
}
```

```

    at[i] = true;
    if (bt[i] == 0)
        tot[i] = at[i] - at[i];
    at[i] = tot[i] - bt[i];

```

```

    complete++;
    }
    }
    free++;
}

```

```

    printf("\n PJP \t AT \t BT \t CT \t WT \t T1
           \t RT \t n");

```

```

    for (int i = 0; i < n; i++) {
        printf("%d \t %d \t %d \t %d \t %d \t %d \t
               %d \n", i+1, at[i], bt[i], ct[i], wt[i],

```

```

               rt[i], tot[i]);
        tot = wt[i];
    }

```

```

    }
    }
    }

```

```

    printf("\n Avg WT = %.2f \n Avg TAT = %.2f \n
           Avg RT = %.2f \n Ave/n, d/n, b/n, c/n,

```

int main() {

int n, choice, v;
int at[100], bt[100], pn[100];

printf("Processes : ");
scanf("%d", &n);

for(int i=0; i<n; i++) {

printf("P%d AT BT PR", i+1);
scanf("%d %d %d", &at[i], &bt[i], &pn[i]);

printf("P%d Priority %d - Preemptive");

printf("P%d Priority %d - Non-preemptive");

printf("P%d Round Robin");

printf("P%d Priority %d - Preemptive");

scanf("%d", &choice);

switch (choice) {

case 1:

priority_np { n, at, bt, pn };
break;

Case 2:
 Priority - P (a, at, bt, pa),
 back;

Case 3:
 Priority ("Quantum's"),
 each (1, 2, 3, 4, 5),
 run (n, at, bt, v);
 back;

return 0;

Output:

Enter number of processes: 5
 Enter AT BT PR for P1: 0 3 5
 Enter AT BT PR for P2: 2 2 3
 Enter AT BT PR for P3: 3 5 2
 Enter AT BT PR for P4: 4 4 4
 Enter AT BT PR for P5: 6 1 1

1. Priority the Receiver
2. Priority Receiver

3. Round Robin

Kid's choice : 1

GRANT Chart :

1	P ₁	1	P ₃	1	P ₅	1	P ₂	1	P ₄	1
0		3		8		4		4		15

PID	AT	BT	CT	WT	TAT	RT
1	0	3	3	0	3	0
2	2	2	11	7	9	4
3	3	5	8	0	5	0
4	4	4	15	7	11	7
5	6	1	9	2	3	2

Aug WT : 3.20
 Aug TAT : 6.00 ~~10.10~~
 Aug RT : 3.20

Kid's choice : 2

GRANT Chart :

1	P ₁	1	P ₂	1	P ₃	1	P ₅	1	P ₈	1	P ₂	1	P ₄	1	P ₁	1
0		2		3		6		7		9		10		14		15

PID	AT	BT	P _q	CT	TAT	WT	RT
P ₁	0	3	5	15	15	12	0
P ₂	2	2	8	10	8	16	0
P ₃	3	5	2	9	6	1	0
P ₄	4	4	4	14	10	6	6
P ₅	6	1	1	7	1	0	0

Aug WT = 5.00

Aug TAT = 8.00

Aug RT = 10.20

Enter choice 3

GRANT Chart:

P ₁	P ₂	P ₃	P ₄	P ₅	P ₃
3	5	9	13	14	16

PID	AT	BT	CT	TAT	WT	RT
1	0	0	3	3	0	0
2	2	3	5	1	1	1
3	3	5	10	13	8	2
4	4	4	13	1	5	5
5	6	1	14	8	4	7

Aug wt = 4.20
Aug AT = 4.20
Aug FI = 3.00

~~7/19/25~~

Multithreaded Queue (Preemptive)

#include <stdio.h>
#define MAX 100

typedef struct {
int id, at, bt, rt, ct, wt, hf, type;
} Process;

Process p[MAX];
int n;
int q[100], rear = 0, front = 0;
int head = 0, rear = 0;
int tail = 0, rear = 0;

void enqueue(int q[], int *rear, int val) {
q[(rear++) % 100] = val;
}

int dequeue(int q[], int *front, int *rear) {
if (*front == *rear) return -1;
return q[(*front++) % 100];
}

int dequeue(int front, int rear) {
return front == rear ? -1 : front;
}

int main() {

printf("Enter number of processes: ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
printf("Enter process %d\n", i);
printf("Enter arrival time: ");
scanf("%d", &p[i].at);
printf("Enter burst time: ");
scanf("%d", &p[i].bt);
printf("Enter type (0 = system, 1 = user): ");
scanf("%d", &p[i].type);
}

p[i].id = i;
p[i].rt = p[i].bt;

for (int i = 0; i < n-1; i++)
if (p[i].at > p[i+1].at)
{
Process temp = p[i];
p[i] = p[i+1];
p[i+1] = temp;
}

```

    p[i] = p[j];
    p[j] = temp;
}

int time = 0, completed = 0, i = 0;
int current = -1;
int gantt[MAX], gtime[MAX], gindex = 0;

while (completed < n) {
    while (i < n && p[i].at <= time) {
        if (p[i].type == 0)
            enqueue(sysQ, &rearS, i);
        else
            enqueue(userQ, &rearU, i);
        i++;
    }
    if (current != -1 && p[current].type == 1 &&
        !isEmpty(frontS, rearS)) {
        enqueue(userQ, &rearU, current);
        current = -1;
    }
    if (current == -1) {
        if (!isEmpty(frontS, rearS))
            current = dequeue(sysQ, &frontS, &rearS);
        else if (!isEmpty(frontU, rearU))

```

Date: / /
Page:

```

        current = dequeue(userQ, &frontU, &rearU);
    } else {
        time++;
        continue;
    }
}

gantt[gindex] = p[current].id;
gtime[gindex] = time;
gindex++;
p[current].rt--;
time++;

if (p[current].rt == 0) {
    p[current].ct = time;
    p[current].tat = p[current].ct - p[current].at;
    p[current].bt = p[current].tat - p[current].bt;
    completed++;
    current = -1;
}

gtime[gindex] = time;
printf("\n Gantt Chart: \n");
for (int i = 0; i < gindex; i++) {
    printf("P%d ", gantt[i]);
}

```

```

int qindex = 0;
printf("\n0");
for (int i = 0; i < qindex; i++) {
    printf(" %d", gtime[i+1]);
}
}

```

float totalWT = 0, totalTAT = 0;

```

printf("\n\nID\tType\tAT\tBT\tCT\n\t\tWT\tTAT\n");
for (int i = 0; i < n; i++)

```

```

{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           i, p[i].type, p[i].at, p[i].bt, p[i].ct,
           p[i].wt, p[i].tatt);
    totalWT += p[i].wt;
    totalTAT += p[i].tatt;
}

```

```

printf("\n\nAverage Waiting Time: %.2f",
       totalWT/n);
return 0;
}

```

```

printf("\n\nAverage Waiting Time: %.2f",
       totalWT/n);
return 0;
}

```

Output:

Enter number of processes: 3

Process 0

Enter arrival time: 0

Enter burst time: 3

Enter type (0=System, 1=User): 0

Process 1

Enter arrival time: 3

Enter burst time: 4

Enter type (0=System, 1=User): 1

Process 2

Enter arrival time: 4

Enter burst time: 2

Enter type (0=System, 1=User): 0

Gantt Chart:

1	P0	1	P0	1	P0	1	P1	1	P2	1	P2	1	P2	1	P2	1	P2	1	P2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

ID	Type	AT	BT	CT	WT	TAT
0	System	0	3	3	0	3
1	User	3	4	4	3	6
2	System	4	2	6	0	2

Average Waiting Time: 0.67

Average Turnaround Time: 3.67

Lab 6: RM, EDF, Proportional

```
#include <stdio.h>
#include <math.h>
#define MAX 10

int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int RM/M(int arr[], int n) {
    int sum = arr[0];
    for (int i = 1; i < n; i++)
        sum = (sum * arr[i]) / gcd(sum, arr[i]);
    return sum;
}
```

```
float calcUtil(int n, int exec[], int period[]) {
    float u = 0;
    for (int i = 0; i < n; i++)
        u += (float) exec[i] / period[i];
    return u;
}
```

```
void printGantt(int t[], int n) {
    printf("\n Gantt Chart: \n");
    for (int i = 0; i < n; i++)
        printf("%d ", t[i]);
}
```

```
printf("\n 0");
for (int i = 1; i <= n; i++)
    printf("%d ", i);
printf("\n");
```

```
void printTable(int n, int exec[], int period[],
               int deadline[], int type) {
    printf("\n Final Table: \n");
    if (type == 2)
        printf("Task \t Start \t T \t TAT \t WT \n");
    else if (type == 1)
        printf("Task \t Exec \t Period \t Deadline \t T \n");
}
```

```
for (int i = 0; i < n; i++) {
    int lat = dt[i];
    int wt = lat - exec[i];
    if (type == 2)
        printf("T %d \t %d \t %d \t %d \t %d \n",
               i+1, exec[i], dt[i], lat, wt);
}
```

```
float lres = calcUtil(n, exec, period);
float bound = n * (prob(2, 1.0/n) - 1);
```

```

if (u > bound) {
    printf("Not schedulable");
    return u;
}

printf("Schedulable under RM\n");
int limit = find_LCM(periods, n);
for (int i = 0; i < n; i++) remaining[i] = 0;
for (int time = 0; time < limit; time++) {
    int s = -1, min = 9999;
    for (int i = 0; i < n; i++) {
        if (time % period[i] == 0)
            remaining[i] = exec[i];
        for (int j = 0; j < n; j++) {
            if (remaining[j] > 0 && period[j] < min)
                min = period[j], s = j;
        }
        if (s != -1) {
            time[s] = s + 1;
            remaining[s]--;
        }
        if (remaining[s] == 0) {
            printf("Task %d", s + 1);
            if (s != n - 1) printf(" ");
        }
    }
    printf("\n");
}

```

Date: / / Page:

```

void edf() {
    int n, exec[100], period[100],
        deadline[100];
    int timeline[500], remaining[100],
        abs_dead[100], t[100] = {0};
    printf("Enter no. of tasks: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Task %d Execution time: ", i + 1);
        scanf("Task %d Period: ", &i);
        printf("Task %d Deadline: ", i + 1);
        scanf("Task %d Deadline: ", &i);
    }
    float u = calculate_utilization(period);
    printf("In CPU utilization %f", u);
    if (u > 1) {
        printf("Not Schedulable under EDF\n");
        return;
    }
    printf("Schedulable\n");
    int limit = find_LCM(period, n);
    for (int i = 0; i < n; i++) remaining[i] = 0;
    abs_dead[i] = deadline[i];
    for (int time = 0; time < limit; time++) {
        int s = -1, min = 9999;
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0)
                remaining[i] = exec[i];
            for (int j = 0; j < n; j++) {
                if (remaining[j] > 0 && period[j] < min)
                    min = period[j], s = j;
            }
            if (s != -1) {
                time[s] = s + 1;
                remaining[s]--;
            }
            if (remaining[s] == 0) {
                printf("Task %d", s + 1);
                if (s != n - 1) printf(" ");
            }
        }
        printf("\n");
    }
}

```

```

for (int i = 0; i < n; i++)
    if (time % period[i] == 0) {
        remaining[i] = exec[i];
        abs-dead[i] = time + deadline[i];
    }

```

```

void proportional() {
    int n, share[MAX], timeline[500],
        t[MAX] = 0;
    int total = 0;
    for (int i = 0; i < n; i++) {
        printf("Task %d share: ", i+1);
        scanf("%d", &share[i]);
        total += share[i];
    }

```

```

    int limit = total, time = 0;
    while (time < limit) {
        for (int i = 0; i < n & time < abs-dead[i];
            i++) {
            for (int j = 0; j < share[i] & time < limit;
                j++) {
                timeline[time] = i+1;
                time++;
                if (time == abs-dead[i] - 1) abs-dead[i] = time + j;
            }
        }
    }
}

```

Date: / /
Page:

```

int main() {
    int ch;
    printf("1. RM, 2. EDF, 3. Proportional\n");
    printf("choice: ");
    scanf("%d", &ch);

    if (ch == 1) rateMonotonic();
    else if (ch == 2) edf();
    else if (ch == 3) proportional();
    else printf("Invalid\n");
}

```

O/Ps

1. RM

2. EDF

3. Proportional

choice: 1

Enter number of tasks: 3

Task 1 Execution Time: 5

Task 1 Period: 5

Task 2 Execution Time: 3

Task 2 Period: 20

Task 3 Execution Time: 2

Task 3 Period: 10

CPU Utilization = 0.350

Bound = 0.780

Not schedulable

Choice 2

No. of Tanks: 3

T1 ET: 3

T1 Period: 20

T1 Deadline: 7

T2 ET: 2

T2 Period: 5

T2 Deadline: 4

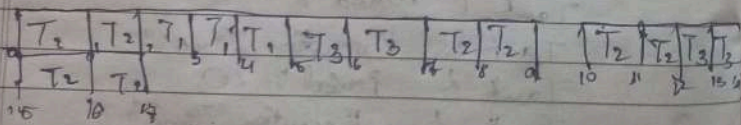
T3 ET: 2

T3 Period: 10

T3 Deadline: 8

CPU Utilization = 0.750

Gantt Chart



Dining Philosopher

The following program controls of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1, S_1 = 0, S_2 = 0$. Find out how many times Process P_0 will print "0". Assume the order of execution as P_0, P_1, P_0, P_1 .

Process P_0 :

while (true) {

wait(S_0);

print("0");

signal(S_1);

signal(S_2);

}

Process P_1 :

wait(S_1);

signal(S_0);

Process P_2 :

wait(S_2);

signal(S_0);

Ans	Initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	Print
P_0	0	0	1	1	0
P_1	1	1	0	1	-
P_2	1	1	0	0	-
P_0	0	0	1	1	00
P_1	1	1	0	1	-
P_2	1	1	0	0	-

P_0 will print 0 Two times

Consider two processes A and B. Two semaphore variables S and T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled as A B A B A B A B A B. What will be the output?

Process A:

```
while (1)
wait(S);
print P;
signal(S);
```

Process B:

```
while (1)
wait(T);
print I;
signal(T);
```

Initial	S=0	T=1	Remark
A	0	1	A is block
B	1	0	print II
A	0	1	print PP
B	1	0	print II
B	1	0	B is block
A	0	1	print PP
A	0	1	block A
B	1	0	II
A	0	1	PP

Final O/P - IIPPIIPP IIPP

Producer Consumer

Philosophers

Initial	F ₀	F ₁	F ₂	F ₃	F ₄	Remark
P ₂	1	1	0	0	1	P ₂ Eating
P ₃	1	0	0	0	0	-
P ₀	0	0	0	0	0	P ₀ Eating
P ₁	1	1	0	0	0	P ₁ Thinking
P ₂	1	1	1	1	0	P ₂ Thinking
P ₃	1	1	1	0	0	P ₃ Thinking
P ₁	1	0	0	0	0	P ₁ Eating

Output:

Philosopher 3 is thinking
P1 is thinking
P0 is thinking
P1 is thinking
P2 is thinking
P2 picked up left fork 2
P2 picked up right fork 3
P2 is eating
P3 picked up right fork 4

P0 picked up left fork 1
 P0 is eating
 P0 put down forks 0 and 1
 P0 is thinking
 P2 put down forks 2 and 3
 P2 is thinking
 P3 picked up left fork 3
 P3 is eating
 P1 picked up right fork 2
 P1 picked up left fork 1
 P1 is eating

Code:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5

sem_t sum;
pthread_t philosophers[N];

void *dine(void *num) {
    int id = *(int *)num;
  
```

Date: / /
Page:

```

    int left = id;
    int right = (id+1) % N;

    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0) {
            sem_wait(&sum); // 4 forks [left]
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        } else {
            sem_wait(&sum); // 4 forks [right]
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(1);

        sem_post(&sum); // 4 forks [left]
        sem_post(&sum); // 4 forks [right]
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
  
```

```

int main () {
    int ids[N];
    for (int i=0; i<N; i++)
        sem_init(&forks[i], 0, 1);
    for (int i=0; i<N; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i],
            NULL, dine, &ids[i]);
    }
    for (int i=0; i<N; i++)
        pthread_join(philosophers[i], NULL,
            &ids[i]);
    return 0;
}

```

Producer Consumer:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define max 20

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i=0; i<max; i++) {
        int item = i;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced item at buffer\n [%d] \n", item, i);
    }
}

```

```

in = (in + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex);
sem_post(&full);
usleep(10000);
}
return NULL;

void *consumer(void *arg) {
    for(int i=0; i < max; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[in];
        printf("Consumed: %d from\n", item);
        out = (out + 1) % BUFFER_SIZE;
        return NULL;
    }
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, producer,
        NULL);

```

Date: / /
Page:

```

pthread_join(prod, NULL);
pthread_join(cons, NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
return 0;
}

```

O/P :

```

Enter buffer size : 3
Enter number of items : 4
[Producer] Produced: 1 → Buffer[0]
[Consumer] Consumed: 1 ← Buffer[0]
[Producer] Produced: 2 → Buffer[1]
[Producer] Produced: 3 → Buffer[2]
[Consumer] Consumed: 2 ← Buffer[1]
[Producer] Produced: 4 → Buffer[0]
[Consumer] Consumed: 3 ← Buffer[2]
[Consumer] Consumed: 4 ← Buffer[0]

```

Banks' Safety Algorithm

```
#include <stdio.h>
#include <stdbool.h>
```

```
int main () {
```

```
    int P, R;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &P);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &R);
```

```
    int Allocation[P][R], Max[P][R], Need[P][R];
```

```
    int Total[R], Available[P], work[R];
```

```
    bool Finish[P];
```

```
    int SafeSequence[P];
```

```
    printf("\n Enter Allocation Matrix: \n");
```

```
    for (int i = 0; i < P; i++) {
```

```
        printf("Process P %d: \n", i);
```

```
        for (int j = 0; j < R; j++) {
```

```
            scanf("%d", &Allocation[i][j]);
```

```
        }
```

```
    }
```

Date / /
Page

```
    printf("\n Enter Total Resources: \n");
    for (int j = 0; j < R; j++) {
        scanf("%d", &Total[j]);
    }
```

```
    // Calculate available
```

```
    for (int j = 0; j < R; j++) {
```

```
        int sum = 0;
```

```
        for (int i = 0; i < P; i++) {
```

```
            sum += Allocation[i][j];
```

```
        }
```

```
        Available[j] = Total[j] - sum;
```

```
        Work[j] = Available[j];
```

```
    }
```

```
    for (int i = 0; i < P; i++) {
```

```
        for (int j = 0; j < R; j++) {
```

```
            Need[i][j] = Max[i][j] - Allocation[i][j];
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < P; i++) {
```

```
        Finish[i] = false;
```

```
    }
```

Safety Algorithm

```

while (count < P) {
    bool found = false;
    for (int i = 0; i < P; i++) {
        if (!Finish[i]) {
            int j;
            for (j = 0; j < R; j++) {
                if (Need[i][j] > work[j])
                    break;
            }
            if (j == R) {
                for (int k = 0; k < R; k++)
                    if (work[k] + Allocation[i][k] > Max[k])
                        break;
                if (k == R)
                    found = true;
            }
        }
    }
    if (!found) {
        printf("The System is in Unsafe State\n");
        return 0;
    }
    printf("Safe State\n");
    printf("Safe sequence:");
    for (int i = 0; i < P; i++) {
        printf("P%d", SafeSequence[i]);
        printf("\n");
    }
    return 0;
}
    
```

O/P:

Enter no. of processes: 5
 Enter no. of resource types: 3
 Enter Allocation matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter Max matrix:

7	5	3
3	4	2
9	0	2
2	2	2
4	3	3

Enter available resource
 4 2 5

Safe Sequence: P1 P3 P4 P0 P2

```

bool isSafeState(int P, int R, int Allocation[P][R],
    int Need[P][R], int Available[R],
    int SafeSequence[P]) {

```

```

    int Work[R]

```

```

    bool Finish[P];

```

```

    for (int i = 0; i < R; i++)

```

```

        Work[i] = Available[i];

```

```

    for (int i = 0; i < P; i++)

```

```

        Finish[i] = false;

```

```

    int count = 0;

```

```

    while (count < P) {

```

```

        bool found = false;

```

```

        for (int i = 0; i < P; i++) {

```

```

            if (!Finish[i]) {

```

```

                int j;

```

```

                for (j = 0; j < R; j++) {

```

```

                    if (Need[i][j] > Work[j])

```

```

                        break;

```

```

                }

```

```

            if (j == R) {

```

```

                for (int k = 0; k < R; k++)

```

```

                    Work[k] += Allocation[i][k];

```

```

            }

```

```

            SafeSequence[count++] = i;

```

```

            Finish[i] = true;

```

```

        }
    }
    found = true;
}

```

if (found)

return false;

return true;

Output:

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation

Matrix:

Enter Max Matrix

Process P0:

Process P0:

0 1 0

4 5 3

Process P1:

Process P1:

2 0 0

3 2 2

Process P2:

Process P2:

3 0 2

9 0 2

Process P3:

Process P3:

2 1 1

2 2 2

Process P4:

Process P4:

0 0 2

4 5 5

Enter Total Resources:

10 5 7

System is in SAFE state.

Safe Sequence: P1 P3 P4 P0 P2

If you want to make a resource request

Enter process number: 4 Enter request vector: 3 3 0

Request denied. System would be unsafe.

```
#include <stdio.h>
#define MAX 10
```

```
int main() {
    int n, m;
    int alloc[MAX][MAX], request[MAX][MAX];
    int avail[MAX];
    int finish[MAX];
    int i, j, k;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource types: ");
scanf("%d", &m);
```

```
printf("Enter allocation Matrix: ");
for (i = 0; i < n; i++) {
    printf("P%d: ", i);
    for (j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}
```

```
for (i = 0; i < n; i++) {
    int hasAllocation = 0;
```

```
for (j = 0; j < m; j++) {
    if (alloc[i][j] != 0) {
        hasAllocation = 1;
        break;
    }
}
```

```
if (hasAllocation == 0)
    finish[i] = 1;
else
    finish[i] = 0;
```

```
int found;
```

```
do {
    found = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            for (j = 0; j < m; j++) {
                if (request[i][j] > avail[j])
                    break;
            }
        }
    }
```

```
if (j == m) {
    for (k = 0; k < m; k++) {
        avail[k] += alloc[i][k];
    }
    finish[i] = 1;
    found = 1;
}
```

printf("In Process P%d has completed\n", i);

if (i == 0)

{ while (found);

int deadlock = 0;

printf("In --- Result ---\n");

for (i = 0; i < n; i++) {

if (finish[i] == 0) {

printf("Process P%d is in
Deadlock\n", i);

deadlock = 1;

}

if (deadlock == 0) {

printf("No deadlock detected.

System is safe.\n");

return 0; }

O/P:

Enter number of processes: 4

Enter number of resource types: 3

Enter Allocation Matrix:

P0: 1 0 2

P1: 2 1 1

P2: 1 0 3

P3: 1 2 2

Enter Request Matrix:

P0: 0 0 1

P1: 1 0 2

P2: 0 0 0

P3: 3 3 0

Enter Available Resources:

0 0 0

Process P2 has completed

Process P0 has completed

Process P1 has completed

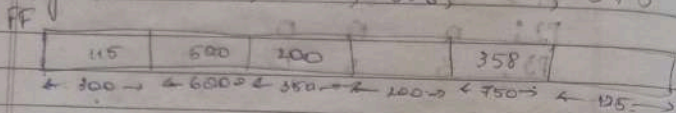
--- Result ---

Process P3 is in Deadlock

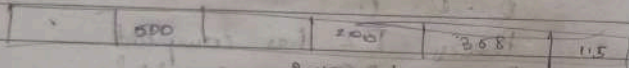
5/5/20

Memory partitions $\rightarrow$ 300 KB, 600 KB, 350 KB,
200 KB, 450 KB, 125 KB

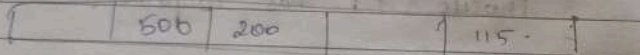
Algorithms $\rightarrow$ 115, 500, 358, 200, 375



BF



WF



358 & 375 can't be placed

Contiguous Memory Allocation Technique

#include <stdio.h>

void firstFit (int blocks[], int m, int processes[],
int n) {

int allocation[n];

for (int i = 0; i < n; i++)

allocation[i] = -1;

for (int j = 0; j < m; j++)

for (int k = 0; k < n; k++)

if (blocks[j] >= processes[k])

allocation[i] = j;
blocks[j] -= processes[i];
break;

void bestFit (int blocks[], int m, int processes[],
int n) {

int allocation[n];

for (int i = 0; i < n; i++)

allocation[i] = -1;

for (int i = 0; i < n; i++)

int best = -1;

for (int j = 0; j < m; j++)

if (blocks[j] >= processes[i])

if (bestIdx == -1 || blocks[j] < blocks[bestIdx])

bestIdx = j;

void worstFit (int blocks[], int m, int processes[],
int n) {

int allocation[n];

for (int i = 0; i < n; i++)

allocation[i] = -1;

```

for (int i = 0; i < n; i++) {
    int worstIdx = -1;
    for (int j = 0; j < m; j++) {
        if (blocks[j] > process[i])
            if (worstIdx == -1 || blocks[j] > blocks[worstIdx])
                worstIdx = j;
    }
    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        blocks[worstIdx] -= process[i];
    }
}

```

```

int main() {
    int m, n;
    printf("Enter number of memory blocks:");
    scanf("%d", &m);

    int blocks[m], blocks1[m], blocks2[m], blocks3[m];
    printf("Enter sizes of %d memory blocks: \n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d", &blocks[i]);
        blocks1[i] = blocks[i];
        blocks2[i] = blocks[i];
        blocks3[i] = blocks[i];
    }
}

```

Date: / /
Page:

```

printf("Enter number of process:");
scanf("%d", &n);

int processes[n];
printf("Enter sizes of %d processes: \n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

printf("Best Fit: blocks 1, m, processes, n");
printf("Worst Fit: blocks 2, m, processes, n");
printf("Worst Fit: blocks 3, m, processes, n");
return 0;
}

```

O/P :- First Fit

1	212	1
2	512	2
3	312	5
4	524	Not allowed

Best Fit			Worst Fit		
1	212	4	1	212	2
2	512	5	2	512	5
3	312	1	3	312	1
4	526	2	4	526	Not allowed

#4 Page Replacement Algorithms: (i) LIFO
(ii) LRU
(iii) Optimal

```
#include <stdio.h>
int findLRU(int time[], int n) {
    int i, min = time[0], pos = 0;
    for (i = 1; i < n; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
}
```

return pos;

```
int main() {
    int pages[30], frames[10], time[10];
    int n, i, j, k, faults, counts;
    printf("Enter the number of frames");
    scanf("%d", &n);
    printf("Enter page reference string");
```

Date: / /
Page:

```
for (i = 0; i < n; i++)
    scanf("%d", &pages[i]);
printf("Enter no. of frames");
scanf("%d", &j);
```

```
for (i = 0; i < j; i++) frames[i] = 0;
faults = 0; pos = 0;
printf("In FIFO Page Replacement");
for (j = 0; j < n; j++) {
    if (frames[j] == pages[i])
        found = 1;
    if (!found) {
        frames[pos] = pages[i];
        pos = (pos + 1) % j;
        faults++;
    }
}
```

```
for (i = 0; i < j; i++) frames[i] = 0;
faults = 0; counter = 0;
printf("In LRU Page Replacement");
for (i = 0; i < n; i++) {
    found = 0;
    for (j = 0; j < j; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            counter = counter + 1;
        }
    }
}
```

```

- printf("Page %d -> ", pages[i]);
for (k=0; k < f; k++)
    frames[k] == -1 ? printf("%d", frames[k]) : printf("-");
printf("\n");

```

```

printf("Total page faults (FIFO) = %d\n", faults);

```

```

for (i=0; i < f; i++) frames[i] = -1;
faults = 0; counter = 0;
printf("In LRU page replacement\n");

```

```

for (i=0; i < n; i++) {
    found = 0;
    for (j=0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            counter++;
            found = 1;
            frames[j] = counter;
            break;
        }
    }
}

```

```

if (!found) {
    if (i < f)
        frames[i] = pages[i];
}

```

Date: / /
Page: /

```

else {
    pos = findLRU(time, f);
    frames[pos] = pages[i];
    counter++;
    found = 1;
    faults++;
}

```

```

printf("Pages %d is ", pages[i]);
for (k=0; k < f; k++)
    frames[k] == -1 ? printf("%d", frames[k]) : printf("-");
printf("\n");

```

```

printf("Total page faults (LRU) = %d\n", faults);

```

```

for (i=0; i < f; i++) frames[i] = -1;
faults = 0;
printf("In Optimal Page Replacement\n");

```

```

for (i=0; i < n; i++) {
    found = 0;
    if (!found) {
        if (i < f)
            frames[i] = pages[i];
    }
}

```

```

else {
    int future[10], j, k;
}

```

```

for (k = 1; k <= n; k++) {
    if (frames[j] == pages[k]) {
        future[j] = k;
        flag[j] = 1;
        break;
    }
}

```

```

frames[pos] = pages[k];
}

```

```

faults++;

```

```

printf("Page %d -> ", pages[i]);

```

```

for (k = 0; k < f; k++)

```

```

{
    if (frames[k] != -1) printf("%d ", frames[k]);
    printf("\n");
}

```

```

printf("\n");

```

```

printf("Total Page faults - optional = %d\n", faults);

```

O/P: Enter number of pages: 20

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter frame size: 3

Date: / /
Page:

Output:

FIFO:

7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1

0 -> 2 3 1

3 -> 2 3 0

0 -> 4 3 0

4 -> 4 2 0

2 -> 4 2 3

3 -> 0 2 3

0 -> 0 1 3

3 -> 0 1 2

2 -> 0 2 3

1 -> 0 1 3

2 -> 0 1 2

0 -> 0 1 2

1 -> 7 1 2

7 -> 7 0 2

0 -> 7 0 2

1 -> 7 0 1

Total page faults = 15

LRU:

7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1

0 -> 2 0 1

3 -> 2 0 3

0 -> 2 0 3

4 -> 4 0 3

2 -> 4 0 2

3 -> 4 0 2

0 -> 4 3 2

3 -> 0 3 2

2 -> 0 3 2

1 -> 0 3 2

2 -> 1 3 2

0 -> 1 0 2

1 -> 1 0 2

7 -> 1 0 7

0 -> 1 0 7

1 -> 1 0 7

Total page faults = 12

Optimal

7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1

0 -> 2 0 1

3 -> 2 0 3

0 -> 2 0 3

4 -> 2 4 3

2 -> 2 4 3

3 -> 2 4 3

0 -> 2 0 3

3 -> 2 0 3

2 -> 2 0 3

1 -> 2 0 1

2 -> 2 0 1

0 -> 2 0 1

1 -> 2 0 1

7 -> 7 0 1

0 -> 7 0 1

1 -> 7 0 1

Total page faults = 9